

EXPERIMENT 1

Student Name: ARNAV SHARMA

Branch: AIT-CSE-AIML

Semester: 5

Subject Name: ADBMS

UID: 24BAI70781

Section/Group: 24AIT_KRG1_B

Subject Code: 24CSP-310

QUESTION A):

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. "Low Salary": All monthly salaries strictly less than \$20,000.
2. "Average Salary": All monthly salaries in the inclusive range [\$20,000, \$50,000].
3. "High Salary": All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

SOLUTION:

QUERY:

```
INSERT INTO Accounts (account_id, income) VALUES
(3, 108939),
(2, 12747),
(8, 87709),
(6, 91738),
(7, 45169);
```

```
SELECT 'LOW CATEGORY' AS CATEGORY,
COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
FROM ACCOUNTS
WHERE INCOME <20000
```

```
UNION ALL
```

```
SELECT 'AVERAGE CATEGORY' AS CATEGORY,
COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
FROM ACCOUNTS
WHERE INCOME >20000 AND INCOME <=50000
```

```
UNION ALL
```

```
SELECT 'HIGH CATEGORY' AS CATEGORY,
COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
FROM ACCOUNTS
WHERE INCOME >50000;
```

EXPERIMENT 1

OUTPUT:

Data Output Messages Notifications			
			
			SQL
	category text	account_count bigint	
1	LOW CATEORY		1
2	AVERAGE CATEO...		1
3	HIGH CATEORY		3

QUESTION (B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

SOLUTION:

QUERY:

EXPERIMENT 1

Query Query History

```

7      emp_id INT PRIMARY KEY,
8      name VARCHAR(50) NOT NULL,
9      dept_id INT REFERENCES Departments(dept_id)
10     );
11
12     CREATE TABLE Salaries (
13         emp_id INT PRIMARY KEY REFERENCES Employees(emp_id) ON DELETE CASCADE,
14         salary INT NOT NULL
15     );
16
17
18     INSERT INTO Departments (dept_id, dept_name) VALUES
19     (10, 'HR'), (20, 'IT'), (30, 'Sales');
20
21     INSERT INTO Employees (emp_id, name, dept_id) VALUES
22     (1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20), (5, 'Emma', 30);
23
24     INSERT INTO Salaries (emp_id, salary) VALUES
25     (1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);
26
27     SELECT E.*, D.*, S.*
28     FROM
29     EMPLOYEES AS E
30     JOIN
31     DEPARTMENTS AS D
32     ON
33     E.DEPT_ID = D.DEPT_ID
34     JOIN
35     SALARIES AS S
36     ON
37     E.EMP_ID = S.EMP_ID;
  
```

```

UPDATE SALARIES
SET SALARY = SALARY + SALARY * 0.10
WHERE EMP_ID
IN
(
SELECT E.EMP_ID |
FROM EMPLOYEES AS E
JOIN
DEPARTMENTS AS D
ON
E.DEPT_ID = D.DEPT_ID
WHERE D.DEPT_NAME = 'HR'
)
  
```

EXPERIMENT 1

OUTPUT:

	emp_id integer	name character varying (50)	dept_id integer	dept_id integer	dept_name character varying (50)	emp_id integer	salary integer
1	1	Alice	10	10	HR	1	50000
2	2	Bob	10	10	HR	2	25000
3	3	Charlie	20	20	IT	3	80000
4	4	David	20	20	IT	4	40000
5	5	Emma	30	30	Sales	5	45000

QUESTION (C):

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings.

Every customer must choose exactly 3 different toppings to prepare a pizza.

Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost.

Requirements

Every pizza must contain exactly 3 different toppings.

The same topping cannot appear more than once in a pizza.

Different arrangements of the same toppings should be considered the same pizza.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only one should appear.

Display the topping names separated by commas.

Display the total ingredient cost of each pizza.

Sort the output:

First by highest total cost

If two pizzas have the same cost, sort them alphabetically.

EXPERIMENT 1

SOLUTION:

QUERY:

```

1  CREATE TABLE pizza_toppings (
2      topping_name VARCHAR(50) PRIMARY KEY,
3      ingredient_cost NUMERIC(4,2) NOT NULL
4  );
5
6
7  INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES
8      ('Pepperoni', 0.50),
9      ('Sausage', 0.70),
10     ('Chicken', 0.55),
11     ('Onions', 0.25),
12     ('Extra Cheese', 0.40);
13
14  SELECT
15      CONCAT(T1.TOPPING_NAME, ',', T2.TOPPING_NAME, ',', T3.TOPPING_NAME) AS PIZZA,
16      (T1.INGREDIENT_COST+T2.INGREDIENT_COST+T3.INGREDIENT_COST) AS TOTAL_COST
17  FROM PIZZA_TOPPING AS T1
18  JOIN
19  PIZZA_TOPPING AS T2
20  ON
21  T1.TOPPING_NAME < T2.TOPPING_NAME
22  JOIN
23  PIZZA_TOPPING AS T3
24  ON
25  T2.TOPPING_NAME < T3.TOPPING_NAME;

```

OUTPUT:

Data Output			Messages	Notifications
	pizza text	total_cost numeric		
1	Chicken,Extra Cheese,Onions	1.20		
2	Chicken,Extra Cheese,Peppero...	1.45		
3	Chicken,Extra Cheese,Sausage	1.65		
4	Chicken,Onions,Pepperoni	1.30		
5	Chicken,Onions,Sausage	1.50		
6	Chicken,Pepperoni,Sausage	1.75		
7	Onions,Pepperoni,Sausage	1.45		
8	Extra Cheese,Onions,Pepperoni	1.15		
9	Extra Cheese,Onions,Sausage	1.35		
10	Extra Cheese,Pepperoni,Sausa...	1.60		

EXPERIMENT 1

QUESTION (D)

Amazon wants to identify returning customers who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase.

Requirements

1. Compare purchases made by the same user only.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur after the first purchase.
4. Same-day purchases must not be considered.
5. The second purchase must occur within 7 days of the first purchase.
6. Display only the unique User IDs that satisfy the above conditions.
7. Sort the output in ascending order of user_id.

SOLUTION:

QUERY:

```
1 CREATE TABLE amazon_transactions (  
2     id INT PRIMARY KEY,  
3     user_id INT NOT NULL,  
4     item VARCHAR(50),  
5     created_at TIMESTAMP NOT NULL,  
6     revenue INT  
7 );  
8  
9  
10 INSERT INTO amazon_transactions (id, user_id, item, created_at, revenue) VALUES  
11 (1, 101, 'biscuit', '2026-03-01 10:00:00', 12),  
12 (2, 101, 'milk', '2026-03-01 14:00:00', 5),  
13 (3, 101, 'bread', '2026-03-05 09:00:00', 8),  
14 (4, 102, 'banana', '2026-03-10 12:00:00', 4),  
15 (5, 102, 'apple', '2026-03-25 11:00:00', 6),  
16 (6, 103, 'milk', '2026-03-15 08:00:00', 5),  
17 (7, 103, 'bread', '2026-03-16 08:00:00', 7);  
18  
19  
20 SELECT DISTINCT p1.user_id  
21 FROM amazon_transactions p1  
22 JOIN amazon_transactions p2  
23 ON p1.user_id = p2.user_id  
24 AND p1.id != p2.id  
25 AND p2.created_at > p1.created_at  
26 AND p2.created_at :: date - p1.created_at :: date >= 1  
27 AND p2.created_at :: date - p1.created_at :: date <= 7  
28 ORDER BY p1.user_id ASC;
```



OUTPUT:

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	user_id integer 🔒
1	101
2	103